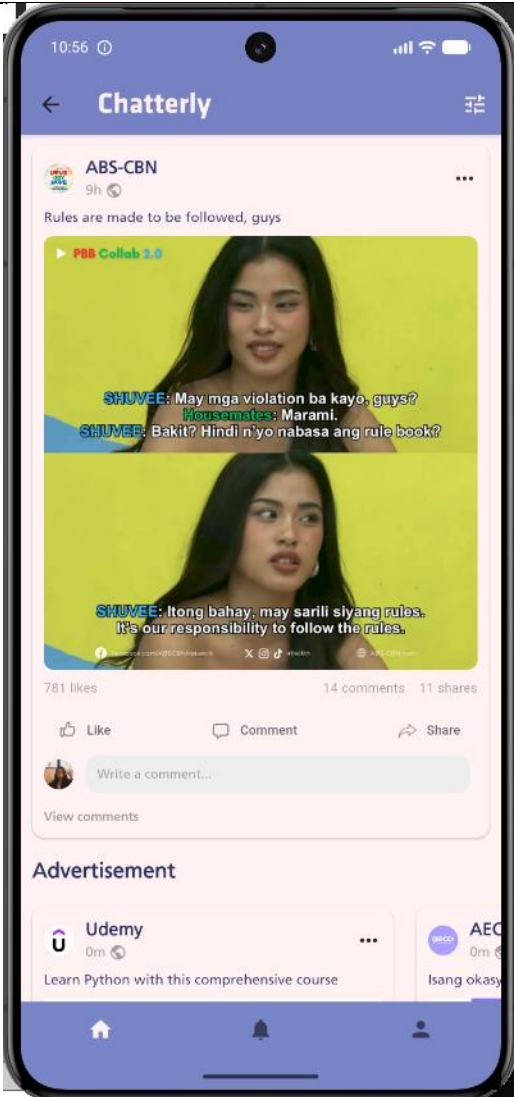


Practical Long Exam Submission Form

Name	Jamaine Grace Tuazon
Section	INF231
Date	August 27, 2026
Place output based on the source code.	





Place your sample code and output based on the enhancement. You may add rows or cells if needed.

Source code – [name of the file]

user_service.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

import '../constants.dart';
import '../models/user.dart';

class UserService {
  Future<User?> login(String username, String password) async {
    final response = await http.post(
      Uri.parse('$host/auth/login'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'username': username, 'password': password}),
    );

    if (response.statusCode == 200) {
      final Map<String, dynamic> data = jsonDecode(response.body);
      return User.fromJson(data);
    }

    return null;
  }

  Future<User?> getUserById(int id) async {
    final response = await http.get(Uri.parse('$host/users/$id'));

    if (response.statusCode == 200) {
      final Map<String, dynamic> data = jsonDecode(response.body);
      return User.fromJson(data);
    }

    return null;
  }

  Future<String?> register({
    required String firstName,
    required String lastName,
    required String phone,
    required String username,
    required String password,
  }) async {
    try {
      final response = await http.post(
        Uri.parse('$host/users/add'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
          'firstName': firstName,
          'lastName': lastName,
          'phone': phone,
          'username': username,
          'password': password,
        })),
      );

      if (response.statusCode == 200 || response.statusCode == 201) {
        return null;
      }
      return 'The server rejected the registration. Status code: '
        '${response.statusCode}';
    } catch (e) {
      return 'Could not reach the server. Please check your internet '
        'connection and try again.';
    }
  }

  Future<Map<int, User>> getAuthors() async {
    final response = await http.get(
      Uri.parse('$host/users?limit=0&select=firstName,lastName,username,image'),
    );

    final Map<int, User> authors = {};
    if (response.statusCode == 200) {
      final Map<String, dynamic> data = jsonDecode(response.body);
      final List usersJson = data['users'] ?? [];
      for (var json in usersJson) {
        final user = User.fromJson(json);
        authors[user.id] = user;
      }
    }
    return authors;
  }
}
```



shared_preference.dart

```
import 'package:shared_preferences/shared_preferences.dart';

import '../models/user.dart';

class SessionService {
  static const String _keyLoggedIn = 'isLoggedIn';
  static const String _keyId = 'userId';
  static const String _keyUsername = 'username';
  static const String _keyEmail = 'email';
  static const String _keyFirstName = 'firstName';
  static const String _keyLastName = 'lastName';
  static const String _keyGender = 'gender';
  static const String _keyImage = 'image';
  static const String _keyPhone = 'phone';
  static const String _keyAge = 'age';
  static const String _keyCity = 'city';
  static const String _keyCountry = 'country';
  static const String _keyCompany = 'company';
  static const String _keyJobTitle = 'jobTitle';
  static const String _keyUniversity = 'university';
  static const String _keyToken = 'token';

  static const String _keyDarkMode = 'darkMode';
  static const String _keyNotifications = 'notifications';

  Future<void> saveUser(User user) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_keyLoggedIn, true);
    await prefs.setInt(_keyId, user.id);
    await prefs.setString(_keyUsername, user.username);
    await prefs.setString(_keyEmail, user.email);
    await prefs.setString(_keyFirstName, user.firstName);
    await prefs.setString(_keyLastName, user.lastName);
    await prefs.setString(_keyGender, user.gender);
    await prefs.setString(_keyImage, user.image);
    await prefs.setString(_keyPhone, user.phone);
    await prefs.setInt(_keyAge, user.age);
    await prefs.setString(_keyCity, user.city);
    await prefs.setString(_keyCountry, user.country);
    await prefs.setString(_keyCompany, user.company);
    await prefs.setString(_keyJobTitle, user.jobTitle);
    await prefs.setString(_keyUniversity, user.university);
    await prefs.setString(_keyToken, user.token);
  }

  Future<bool> isLoggedIn() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_keyLoggedIn) ?? false;
  }

  Future<User?> getUser() async {
    final prefs = await SharedPreferences.getInstance();
    if (!(prefs.getBool(_keyLoggedIn) ?? false)) return null;

    return User(
      id: prefs.getInt(_keyId) ?? 0,
      username: prefs.getString(_keyUsername) ?? '',
      email: prefs.getString(_keyEmail) ?? '',
      firstName: prefs.getString(_keyFirstName) ?? '',
      lastName: prefs.getString(_keyLastName) ?? '',
      gender: prefs.getString(_keyGender) ?? '',
      image: prefs.getString(_keyImage) ?? '',
      phone: prefs.getString(_keyPhone) ?? '',
      age: prefs.getInt(_keyAge) ?? 0,
      city: prefs.getString(_keyCity) ?? '',
      country: prefs.getString(_keyCountry) ?? '',
      company: prefs.getString(_keyCompany) ?? '',
      jobTitle: prefs.getString(_keyJobTitle) ?? '',
      university: prefs.getString(_keyUniversity) ?? '',
      token: prefs.getString(_keyToken) ?? '',
    );
  }

  Future<void> signOut() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_keyLoggedIn, false);
    await prefs.remove(_keyId);
    await prefs.remove(_keyUsername);
    await prefs.remove(_keyEmail);
    await prefs.remove(_keyFirstName);
    await prefs.remove(_keyLastName);
    await prefs.remove(_keyGender);
    await prefs.remove(_keyImage);
    await prefs.remove(_keyPhone);
    await prefs.remove(_keyAge);
    await prefs.remove(_keyCity);
    await prefs.remove(_keyCountry);
    await prefs.remove(_keyCompany);
    await prefs.remove(_keyJobTitle);
    await prefs.remove(_keyUniversity);
    await prefs.remove(_keyToken);
  }

  Future<bool> getDarkMode() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_keyDarkMode) ?? false;
  }

  Future<void> setDarkMode(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_keyDarkMode, value);
  }

  Future<bool> getNotifications() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_keyNotifications) ?? true;
  }

  Future<void> setNotifications(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_keyNotifications, value);
  }
}
```



splash_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';

import '../constants.dart';
import '../services/shared_preference.dart';
import '../widgets/custom_font.dart';

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  final SessionService _session = SessionService();

  @override
  void initState() {
    super.initState();
    _checkSession();
  }

  Future<void> _checkSession() async {
    await Future.delayed(const Duration(seconds: 2));

    final loggedIn = await _session.isLoggedIn();

    if (!mounted) return;
    Navigator.pushReplacementNamed(context, loggedIn ? '/home' : '/login');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: FB_SURFACE,
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Image.asset(
              'assets/images/chatbubble.png',
              height: ScreenUtil().setHeight(200),
            ),
            SizedBox(height: ScreenUtil().setHeight(10)),
            CustomFont(
              text: 'Chatterly',
              fontSize: ScreenUtil().setSp(32),
              color: FB_DARK_PRIMARY,
              fontFamily: 'Klavika',
            ),
          ],
        ),
      ),
    );
  }
}
```

login_screen.dart

```
Future<void> login() async {
  final username = usernameController.text.trim();
  final password = passwordController.text;

  if (username.isEmpty) {
    customDialog(
      context,
      title: 'Invalid Username',
      content: 'Please enter your username.',
    );
    return;
  }
  if (password.isEmpty) {
    customDialog(
      context,
      title: 'Invalid Password',
      content: 'Please enter your password.',
    );
    return;
  }

  setState(() => _isLoading = true);

  try {
    final loggedInUser = await _userService.login(username, password);

    if (loggedInUser == null) {
      if (!mounted) return;
      setState(() => _isLoading = false);
      customDialog(
        context,
        title: 'Login Failed',
        content:
          'Invalid username or password. Please check your credentials '
          'and try again.',
      );
      return;
    }
  }

  final fullProfile = await _userService.getUserById(loggedInUser.id);
  await _session.saveUser(
    fullProfile == null
      ? loggedInUser
      : fullProfile.withToken(loggedInUser.token),
  );

  if (!mounted) return;
  setState(() => _isLoading = false);
  Navigator.pushReplacementNamed(context, '/home');
} catch (e) {
  if (!mounted) return;
  setState(() => _isLoading = false);
  customDialog(
    context,
    title: 'Connection Error',
    content:
      'Could not reach the server. Please check your internet '
      'connection and try again.',
  );
}
}
```



```

        Posts of the signed-in user, fetched by user id
Widget _buildPostsTab() {
  if (!isLoadingPosts) {
    return Padding(
      padding: EdgeInsets.only(top: ScreenUtil().setHeight(30)),
      child: Center(
        child: CircularProgressIndicator(color: FB_LIGHT_PRIMARY),
      ),
    ),
  ),
},

if (_postsError != '') {
  return Padding(
    padding: EdgeInsets.all(ScreenUtil().setSp(20)),
    child: Column(
      children: [
        CustomFont(
          text: _postsError,
          fontSize: ScreenUtil().setSp(14),
          color: FB_TEXT_COLOR_GREY,
        ),
        if (user != null) {
          TextButton(
            onPressed: () => _loadPosts(_userId),
            child: CustomFont(
              text: 'Retry',
              fontSize: ScreenUtil().setSp(14),
              color: FB_DARK_PRIMARY,
              fontWeight: FontWeight.bold,
            ),
          ),
        ],
      ),
    ),
  ),
},

if (_posts.isEmpty) {
  return Padding(
    padding: EdgeInsets.all(ScreenUtil().setSp(20)),
    child: CustomFont(
      text: 'This user has not posted anything yet.',
      fontSize: ScreenUtil().setSp(14),
      color: FB_TEXT_COLOR_GREY,
    ),
  ),
},
},

return ListView.builder(
  padding: EdgeInsets.zero,
  itemCount: _posts.length,
  itemBuilder: (context, index) {
    final post = _posts[index];
    return PostCard(
      postId: post.id,
      postTitle: post.title,
      username: userName,
      postContent: post.body,
      likesCount: post.likes.toString(),
      sharesCount: 0,
      date: DateTime.now().subtract(Duration(days: index + 1)),
      imageUrl: _profileImage,
      currentUserImage: _profileImage,
    ),
  ),
),
},

Widget _buildAboutTab() {
  if (user == null) return const SizedBox();
  final user = _user;

  return SingleChildScrollView(
    child: Padding(
      padding: EdgeInsets.all(ScreenUtil().setWidth(30)),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          SizedBox(height: ScreenUtil().setHeight(10)),
          Text(
            'Description',
            style: TextStyle(
              fontSize: ScreenUtil().setSp(20),
              fontWeight: FontWeight.bold,
              color: FB_TEXT_PRIMARY,
            ),
          ),
          SizedBox(height: ScreenUtil().setHeight(5)),
          Text(
            '$(user.fullName) is a $(user.jobTitle.isEmpty ? "member" : user.jobTitle) '
            '$(user.company.isEmpty ? "" : "at ${user.company}")',
            style: TextStyle(
              fontSize: ScreenUtil().setSp(17),
              color: FB_TEXT_PRIMARY,
            ),
          ),
          SizedBox(height: ScreenUtil().setHeight(20)),
          Text(
            'Details',
            style: TextStyle(
              fontSize: ScreenUtil().setSp(20),
              fontWeight: FontWeight.bold,
              color: FB_TEXT_PRIMARY,
            ),
          ),
          SizedBox(height: ScreenUtil().setHeight(10)),
          _buildDetailRow(
            Icons.house,
            user.city.isEmpty
              ? 'No address on file'
              : 'Lives in ${user.city}, ${user.country}',
          ),
          _buildDetailRow(
            Icons.work,
            user.company.isEmpty
              ? 'No workplace on file'
              : 'Works at ${user.company}',
          ),
          _buildDetailRow(
            Icons.school,
            user.university.isEmpty
              ? 'No school on file'
              : 'Studies at ${user.university}',
          ),
          _buildDetailRow(
            Icons.email,
            user.email.isEmpty ? 'No email on file' : user.email,
          ),
          _buildDetailRow(
            Icons.phone,
            user.phone.isEmpty ? 'No phone on file' : user.phone,
          ),
          _buildDetailRow(
            Icons.cake,
            user.age == 0 ? 'No age on file' : '${user.age} years old',
          ),
        ],
      ),
    ),
  ),
},

Widget _buildDetailRow(IconData icon, String text) {
  return Padding(
    padding: EdgeInsets.only(bottom: ScreenUtil().setHeight(6)),
    child: Row(
      children: [
        Icon(icon, color: Colors.grey, size: ScreenUtil().setSp(17)),
        SizedBox(width: ScreenUtil().setWidth(10)),
        Expanded(
          child: Text(
            text,
            style: TextStyle(
              fontSize: ScreenUtil().setSp(17),
              color: Colors.grey,
            ),
          ),
        ),
      ],
    ),
  ),
},

Widget _buildPhotosTab() {
  return GridView.count(
    primary: false,
    padding: const EdgeInsets.all(10),
    crossAxisSpacing: 2,
    mainAxisSpacing: 2,
    crossAxisCount: 2,
    children: <div>_buildPhotoItem('assets/images/photos1.jpg'),
    _buildPhotoItem('assets/images/photos2.jpg'),
    _buildPhotoItem('assets/images/photos3.jpg'),
    _buildPhotoItem('assets/images/photos4.jpg'),
    _buildPhotoItem('assets/images/photos5.jpg'),
    _buildPhotoItem('assets/images/photos6.jpg'),
    _buildPhotoItem('assets/images/photos7.jpg'),
    _buildPhotoItem('assets/images/coverphoto.jpg'),
    ],
  ),
},

Widget _buildPhotoItem(String imagePath) {
  return Container(
    color: Colors.grey[400],
    child: Image(image: AssetImage(imagePath), fit: BoxFit.cover),
  ),
},
}

```



settings_screen.dart

```
class SettingsScreen extends StatefulWidget {
  const SettingsScreen({super.key});

  @override
  State<SettingsScreen> createState() => _SettingsScreenState();
}

class _SettingsScreenState extends State<SettingsScreen> {
  final SessionService _session = SessionService();

  User? _user;

  @override
  void initState() {
    super.initState();
    _loadSettings();

    Future _loadSettings() async {
      final user = await _session.getSession();

      if (!mounted) return;
      setState(() {
        _user = user;
      });
    }

    Future _onDarkModeChanged(bool value) async {
      context.read<ThemeProvider>().toggleTheme(value);
      await _session.setDarkMode(value);
    }

    Future _signOut() async {
      final confirmed = await showDialog<bool>(
        context: context,
        builder: (context) => AlertDialog(
          title: const Text('Sign out?'),
          content: const Text('Are you sure you want to sign out?'),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context, false),
              child: Text('Cancel', style: TextStyle(color: FB_TEXT_COLOR_GREY)),
            ),
            ElevatedButton(
              style: ElevatedButton.styleFrom(
                backgroundColor: FB_DARK_PRIMARY,
                foregroundColor: Colors.white,
              ),
              onPressed: () => Navigator.pop(context, true),
              child: const Text('Sign out'),
            ),
          ],
        ),
      );
      if (confirmed != true) return;
      await _session.signOut();

      if (!mounted) return;
      Navigator.pushHomeAndRemoveUntil(context, '/login', (route) => false);
    }

    @override
    Widget build(BuildContext context) {
      final themeProvider = context.watch<ThemeProvider>();

      return Scaffold(
        backgroundColor: FB_SURFACE,
        appBar: AppBar(
          backgroundColor: FB_LIGHT_PRIMARY,
          elevation: 2,
          title: const Icon(ThemeData(color: Colors.white),
            title: CustomText(
              text: 'Settings',
              fontSize: ScreenUtil().setSp(25),
              color: FB_TEXT_COLOR_WHITE,
              fontfamily: 'Klavosa',
            ),
          ),
        ),
        body: ListView(
          padding: EdgeInsets.all(ScreenUtil().setSp(15)),
          children: [
            buildAccountCard(),
            buildDarkModeSwitch(),
            buildInfoList(),
          ],
        ),
      );
    }

    Widget _buildAccountCard() {
      final String image = _user == null || _user!.image.isEmpty
        ? assets/images/userprofile.jpg
        : _user!.image;

      return Card(
        color: FB_CARD,
        child: Padding(
          padding: EdgeInsets.all(ScreenUtil().setSp(15)),
          child: Row(
            children: [
              CircleAvatar(
                radius: 20,
                backgroundColor: FB_SECONDARY,
                backgroundImage: Image.startWith('http')
                  ? CachedNetworkImageProvider(image)
                  : AssetImage(image) as ImageProvider,
              ),
              SizedBox(width: ScreenUtil().setWidth(15)),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    CustomText(
                      text: _user?.fullName ?? '',
                      fontSize: ScreenUtil().setSp(12),
                      color: FB_DARK_PRIMARY,
                      fontWeight: FontWeight.bold,
                    ),
                    CustomText(
                      text: _user == null ? '' : '${_user!.username}',
                      fontSize: ScreenUtil().setSp(13),
                      color: FB_TEXT_COLOR_GREY,
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      );
    }

    Widget _buildSectionTitle(String title) {
      return Padding(
        padding: EdgeInsets.only(
          left: ScreenUtil().setWidth(5),
          bottom: ScreenUtil().setHeight(5),
        ),
        child: CustomText(
          text: title,
          fontSize: ScreenUtil().setSp(16),
          color: FB_DARK_PRIMARY,
          fontWeight: FontWeight.bold,
        ),
      );
    }

    Widget _buildDarkModeSwitch({
      required IconData icon,
      required String title,
      required String subtitle,
      required bool value,
      required ValueChanged<bool> onChanged,
    }) {
      return Card(
        color: FB_CARD,
        child: SwitchListTile(
          value: value,
          onChanged: onChanged,
          activeThumbColor: FB_LIGHT_PRIMARY,
          secondary: Icon(icon, color: FB_DARK_PRIMARY),
          title: CustomText(
            text: title,
            fontSize: ScreenUtil().setSp(15),
            color: FB_TEXT_PRIMARY,
          ),
          subtitle: CustomText(
            text: subtitle,
            fontSize: ScreenUtil().setSp(13),
            color: FB_TEXT_COLOR_GREY,
          ),
        ),
      );
    }

    Widget _buildInfoList(IconData icon, String label, String value) {
      return Card(
        color: FB_CARD,
        child: ListTile(
          leading: Icon(icon, color: FB_DARK_PRIMARY),
          title: CustomText(
            text: label,
            fontSize: ScreenUtil().setSp(13),
            color: FB_TEXT_COLOR_GREY,
          ),
          subtitle: CustomText(
            text: value.isEmpty ? 'Not available' : value,
            fontSize: ScreenUtil().setSp(13),
            color: FB_TEXT_PRIMARY,
          ),
        ),
      );
    }
  }
}
```



comments_service.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

import '../constants.dart';
import '../models/comment.dart';

class CommentService {
  Future<List<Comment>> getCommentsByPostId(int postId) async {
    final response = await http.get(Uri.parse('$host/comments/post/$postId'));

    if (response.statusCode == 200) {
      final Map<String, dynamic> data = jsonDecode(response.body);
      final List commentsJson = data['comments'] ?? [];
      return commentsJson.map((c) => Comment.fromJson(c)).toList();
    }
    throw Exception('Failed to load comments (${response.statusCode})');
  }

  required String body,
  required int postId,
  required int userId,
} async {
  final response = await http.post(
    Uri.parse('$host/comments/add'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'body': body, 'postId': postId, 'userId': userId}),
  );

  if (response.statusCode == 200 || response.statusCode == 201) {
    final Map<String, dynamic> data = jsonDecode(response.body);
    return Comment.fromJson(data);
  }
  return null;
}
}
```

newsfeed_screen.dart

```
class NewsfeedScreen extends StatefulWidget {
  const NewsfeedScreen({super.key});

  @override
  State<NewsfeedScreen> createState() => _NewsfeedScreenState();
}

class _NewsfeedScreenState extends State<NewsfeedScreen> {
  final PostService _postService = PostService();
  final UserService _userService = UserService();
  final SessionService _session = SessionService();

  List<Post> _posts = [];
  Map<int, User> _authors = {};
  String _currentUserImage = 'assets/images/userprofile.jpg';
  bool _isLoading = true;
  String _error = '';

  @override
  void initState() {
    super.initState();
    _loadFeed();
  }

  Future<void> _loadFeed() async {
    setState(() {
      _isLoading = true;
      _error = '';
    });

    try {
      final posts = await _postService.getPosts(limit: 12);
      final authors = await _userService.getAuthors();
      final me = await _session.getUser();

      if (!mounted) return;
      setState(() {
        _posts = posts;
        _authors = authors;
        if (me != null && me.image.isNotEmpty) _currentUserImage = me.image;
        _isLoading = false;
      });
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
        _error = 'Could not load the newsfeed. Please try again.';
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    if (_isLoading) {
      return Center(child: CircularProgressIndicator(color: FB_LIGHT_PRIMARY));
    }

    if (_error != '') {
      return Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(Icons.wifi_off, size: ScreenUtil().setSp(40), color: Colors.grey),
            SizedBox(height: ScreenUtil().setHeight(10)),
            CustomFont(
              text: _error,
              fontSize: ScreenUtil().setSp(14),
              color: FB_TEXT_COLOR_GREY,
            ),
            SizedBox(height: ScreenUtil().setHeight(10)),
            TextButton(
              onPressed: _loadFeed,
              child: CustomFont(
                text: 'Retry',
                fontSize: ScreenUtil().setSp(14),
                color: FB_DARK_PRIMARY,
                fontWeight: FontWeight.bold,
              ),
            ),
          ],
        ),
      );
    }

    return RefreshIndicator(
      onRefresh: _loadFeed,
      child: ListView(children: _buildFeedItems()),
    );
  }

  List<Widget> _buildFeedItems() {
    final List<Widget> items = [];

    for (int i = 0; i < _posts.length; i++) {
      final post = _posts[i];
      final author = _authors[post.userId];

      items.add(
        PostCard(
          postId: post.id,
          postTitle: post.title,
          userName: author == null ? 'Chatterly User' : author.fullName,
          postContent: post.body,
          likesCount: post.likes.toString(),
          sharesCount: 0,
          date: DateTime.now().subtract(Duration(hours: i + 1)),
          userImage: (author == null || author.image.isEmpty)
            ? 'assets/images/userprofile.jpg'
            : author.image,
          currentUserImage: _currentUserImage,
        ),
      );

      if (i % 2 == 1) items.add(buildAdvertisementCarousel());
    }

    return items;
  }
}
```



post_card.dart

```
final int postId;

final String postTitle;

final String currentUserImage;
const PostCard({
  super.key,
  required this.userName,
  required this.postContent,
  required this.date,
  required this.userImage,
  this.likesCount = "0",
  this.commentsCount = 0,
  this.sharesCount = 0,
  this.imagePath,
  this.showPlaceholder = false,
  this.adsMarket = '',
  this.postId = 0,
  this.postTitle = '',
  this.currentUserImage = 'assets/images/userprofile.jpg',
});
class _PostCardState extends State<PostCard> {
  final CommentService _commentService = CommentService();
  late int _likes;
  late int _comments;
  bool _isLiked = false;

  @override
  void initState() {
    super.initState();
    _likes = int.tryParse(widget.likesCount) ?? 0;
    _comments = widget.commentsCount;

    // Real posts get their comment count straight from the API.
    if (widget.postId > 0) _loadCommentCount();
  }

  Future<void> _loadCommentCount() async {
    try {
      final comments = await _commentService.getCommentsByPostId(widget.postId);
      if (!mounted) return;
      setState(() => _comments = comments.length);
    } catch (e) {
      // Keep the count that was passed in when the request fails.
    }
  }

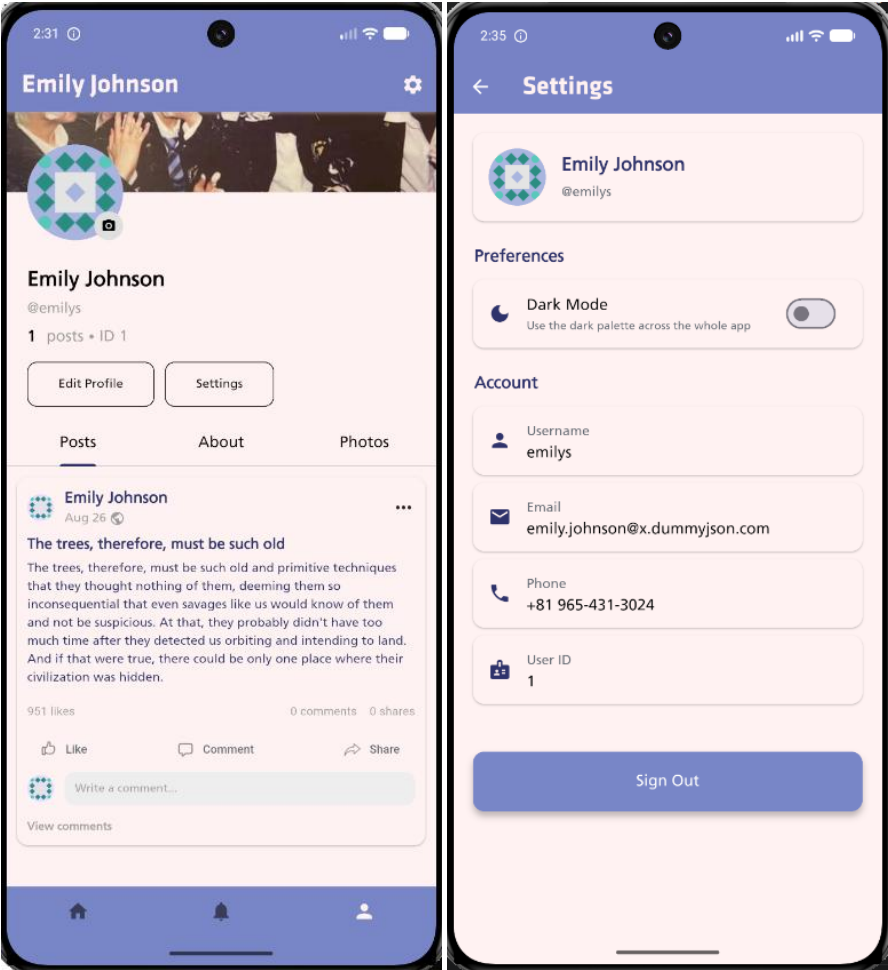
  Future<void> _openDetail() async {
    final result = await Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => DetailScreen(
          postId: widget.postId,
          postTitle: widget.postTitle,
          userName: widget.userName,
          postContent: widget.postContent,
          date: formatDate(widget.date),
          numOfLikes: _likes,
          isLiked: _isLiked,
          imageUrl: widget.imagePath ?? '',
          profileImageUrl: widget.userImage,
          currentUserImage: widget.currentUserImage,
        ),
      ),
    );
    if (!mounted) return;
    if (result != null && result is Map) {
      setState(() {
        if (result['likes'] != null) _likes = result['likes'] as int;
        if (result['isLiked'] != null) _isLiked = result['isLiked'] as bool;
        if (result['comments'] != null) _comments = result['comments'] as int;
      });
    }
  }
  (widget.postTitle == '')
    ? const SizedBox()
    : Padding(
      padding: EdgeInsets.only(
        bottom: ScreenUtil().setHeight(3),
      ),
      child: CustomFont(
        text: widget.postTitle,
        fontSize: ScreenUtil().setSp(14),
        color: FB_DARK_PRIMARY,
        fontWeight: FontWeight.bold,
      ),
    ),
  ),
```

Output – [name of the output]

Enhancement 1: In the documentation <https://dummyjson.com/docs/users> authenticate the user and save the user data using shared_preference. Create a splash_screen to instantiate the mobile application whether the user loggedIn or not.

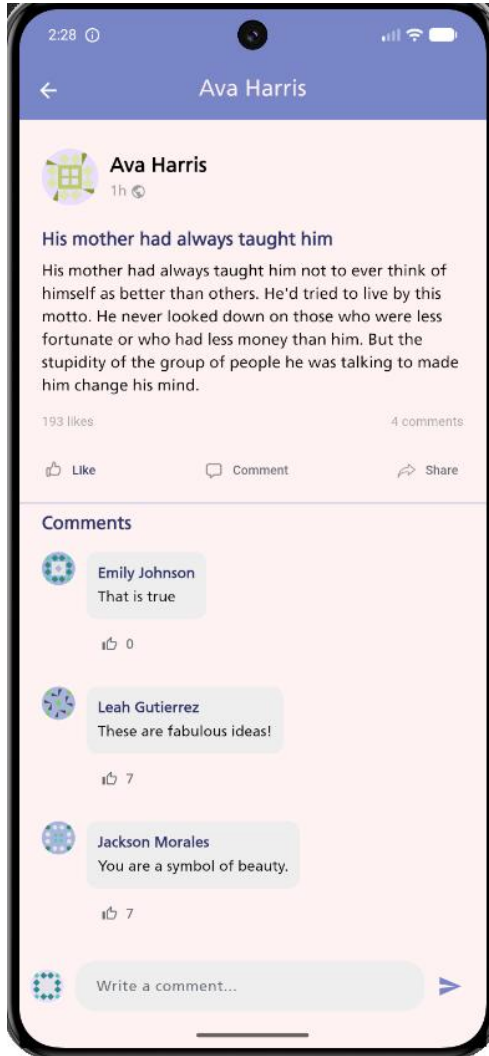
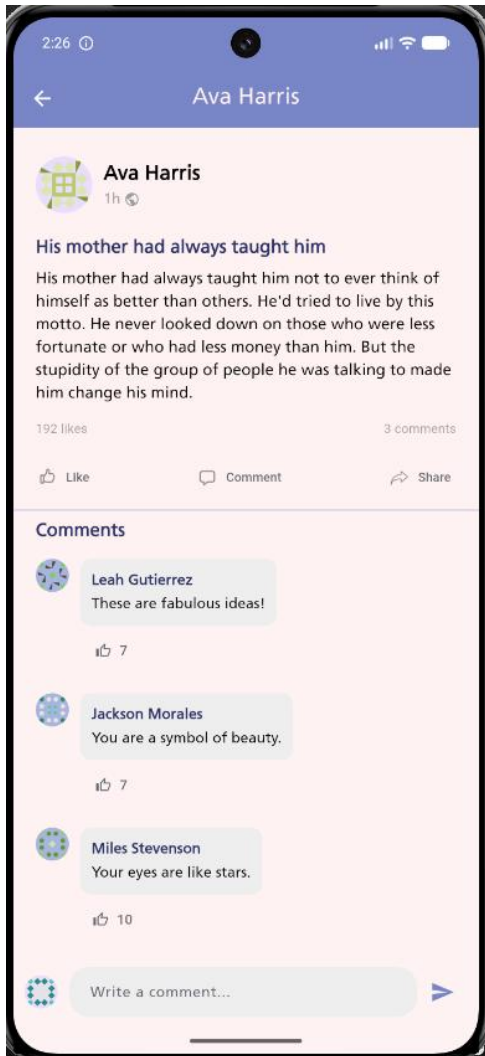


Enhancement 2: In the documentation <https://dummyjson.com/docs/posts> render post on the profile_screen by userID. Create settings screen for user preference as well as Sign Out button.



Enhancement 3: In the documentation <https://dummyjson.com/docs/comments> render all comments

based on each post. The like button must be clickable. You may add comments on each post.



Rubric

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Code Implementation	Code runs flawlessly, includes all required features (placeholders, avatars, widgets for like/comment).	Code mostly correct, minor errors, or missing one minor feature.	Noticeable errors or missing two or more features.	Code does not run or is significantly incomplete.	
Design Consistency	UI closely resembles intended layout, proper spacing, alignment, and styling.	Layout mostly accurate, minor spacing or alignment issues.	Layout deviates significantly from the intended design or appears incomplete.	Layout is unrecognizable or does not follow the instructions.	
Use of Widgets and Enhancements	Properly implements widgets for placeholders, avatars, and like/comment buttons.	Most widgets are correctly implemented, with minor issues or missing one enhancement.	Widgets are inconsistently used or missing two or more enhancements.	Widgets are not used, or enhancements are not implemented.	
Code Organization and Readability	Code is well-structured, follows best practices, and adheres to the specified file organization.	Code is mostly organized and readable, minor deviations from the required structure.	Code is poorly organized or does not follow the specified structure.	Code is disorganized and difficult to understand.	
Back-End Integration	Back-end integration works flawlessly with efficient API calls, secure data handling, and real-time updates; no errors or downtime.	Back-end integration is mostly correct with only minor API, data, or response handling issues.	Back-end integration has noticeable flaws or inefficiencies; multiple API calls fail or data handling is inconsistent.	Back-end integration has major issues; frequent failures, poor response handling, or insecure practices.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/20) * 100	